

COMP 120 Review

Basic Data Types

char
int
float
double

Constants

- Variables that are fixed
const float pi = 3.14;

Type Casting

float salary = 100.26
return (float) salary;

Width and Precision

Specifier	output
%f	24.527810 (9 spots)
%12f	---- 24.527810 (12 spots)
%12.3f	----- 24.528 (12 spots)
%-12.3f	24.528----- (12 spots)

operator	meaning
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
==	Equal to
!=	Not equal to

Loops

```
while (i < SIZE)
{
    // statements
}

do
{
    /* statements */
} while (i < SIZE);

for (i = 0; i < SIZE; i++)
{
    /* statements */
}
```

If/Else

```
if (a % 2 == 0)
    printf("n %d is even", a);
else
    printf("n %d is odd", a);
```

Switch

```
char ch;
switch (ch)
{
    case 'a':
        printf("n %c is a vowel", ch);
        break;
    case 'b':
        printf("n %c isn't a vowel", ch);
        break;
}
```

Break

- Breaks out of the most inner loop its nested in

I/O Functions

```
scanf ("%4d", &num);
```

- Scans the first 4 digits and assigns to num

```
printf ("%9.2f", x);
```

- .2 is digits after decimal
- 9 is width, if greater than x then right justified. If smaller x is printed

Boolean Expressions

*	Multiply
/	Divide
+	Addition
-	Subtraction
%	Modulus // Gives remainder, both must be integers

Operator	meaning
&&	and
	or
!	not

Axioms

$0 \&\& X = 0$
 $1 \&\& X = 1$
 $!0 = 1$

Functions

prototype

function_name (Variable 1, Variable 2);
 for an array:
 int sum (int arr[], int size);

Pointers

int a = 5 // a has value 5
 int *p = &a; // p has value address of a

Dereferencing:

p = address
 *p = value

Generic pointer:

void *gp;
 - must be cast ie:
 printf("n %d, (int)gp);



Pointers

- contains the address of the object

ex: `int a=5;`

in `int* p=&a;`

`p` = mailbox number

`*p = 5`

Dynamic Memory

`malloc()`

- Allocates memory and returns a pointer to the first spot in memory

`calloc()`

- Allocates space and initializes to zero

`free()`

- frees the memory

- watch for dangling pointers

Strings

- a character array terminated by `'\0'`

`int* i;`

`i[0] = 3;`

`*(i+0) = 3;`

- The name of the string is a pointer to the address of the first character

String length

`length = strlen(mystring);`

String compare

`if (strcmp(string1, string2, length) == 0)`

`printf("first length letters are the same")`

`else printf("not the same")`

String Searching

`result = strstr(string2, string1);`

`if result = Null // not found`

String Concatenation

`strcat(a-long-string, "strings");`

Structures

`typedef struct {`

`int empnum;`

`char name[MAXLEN];`

`double salary;`

`} Employee;`

accessing

`variable-name.member`

malloc

`int* i;`

`i = (int*) malloc(10 * sizeof(int))`

`free(i);`

Linked list structures

Topics

- Arrays
- Pointers
- Dynamic Memory
- Strings
- Structures
- Asymptotic Analysis
- Code Analysis

Pointers

```
int *ptr=NULL;
//the value of ptr
is zero
```

How to free dynamic memory

```
int* i = (int*) malloc(sizeof(int));
*i = 5;
free(i);
i = NULL; ← if not, dangling pointer
```

File I/O

```
FILE* filepointer = NULL;
fopen_s(&filepointer, "data.txt", "r");
while (fgets(buffer, SIZE, filepointer);
{
    sscanf(buffer, "%s %s", S1, S2);
}
- fgets goes to EOF, you would have to printf
in the loop but sscanf stores the two strings
separated by a space in S1, S2
```

Dereferencing

```
*(ptr + 20) = 7;
ptr[20] = 7;
```

String functions in <string.h>

```
strcpy(S1, S2)
- changes S1 to S2
- copies S2 into S1
replacing S1
```

```
strcat(S1, S2)
- copies S2 onto
the end of S1
```

```
strlen(S1)
- returns length of
S1
```

```
strcmp(S1, S2);
- returns pointer
to 1: S1 < S2
0: S1 = S2
1: S1 > S2
```

```
strchr(S1, ch)
- returns a pointer
to the first occurrence
of ch in S1
```

```
strstr(S1, S2)
- returns a pointer
to the first occurrence
of S2 in S1
```

Structures

```
- above int main()
```

```
struct Books {
    char title[50];
    char author[50];
    char subject[100];
    int book-id;
};
```

```
//define type Books variables
```

```
struct Books Book1;
struct Books Book2;
```

```
//access by .
```

```
strcpy(Book1.title, "C programming");
Book1.book-id = 25678;
```

*if you pass as a function
you use the function parameter
name. " " to open

Typedef

- Same as struct but allows
you to put in Book1, Book2 without
saying struct

```
typedef struct {
    char title[50];
    char author[50];
    char subject[100];
    int book-id;
} Book;
```

//instead of struct write Book

```
Book Book1;
Book Book2;
```

Pointers to Structures

```
Book* struct-pointer;
struct-pointer = &Book1
//to access inside Book1
struct-pointer->title;
```

Complexity / Analysis

Order of Growth:

Constant (1)
 Logarithmic $O(\log n)$
 Poly-Log $(\log n)^k$
 Linear n
 log-Linear $n \log n$
 Quadratic n^2
 Cubic n^3
 Polynomial n^k
 Exponential C^n

Linked Lists

```
typedef struct node {
    int Val;
    struct node* next;
} node;
```

Create a list:



"goes through each"

```
for (i=0; i<10; i++)
{
    temp->next = (node*) malloc(sizeof(node));
    temp->value = i;
    temp = temp->next;
}
temp->next = NULL;
```

Big O

$0 \leq f(n) \leq C \cdot g(n)$ for $n \geq n_0$
 for some constants C and n_0

Big Ω

$0 \leq C \cdot g(n) \leq f(n)$ for $n \geq n_0$

Big Theta

$C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n)$ for $n \geq n_0$
 different constants

Code Analysis

Linear loops

for (i=0; i<1000; i++)

runs n times $f(n) = n$

Logarithmic loop

for (i=1; i<1000; i*=5)

runs $\log_5 n$ times $f(n) = \log_5 n$

Nested loops

Multiply the loops together

Doubly linked-Lists

```
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
```

Insertion Algorithm:

Node* current, *temp;

① // iterate current into place.....

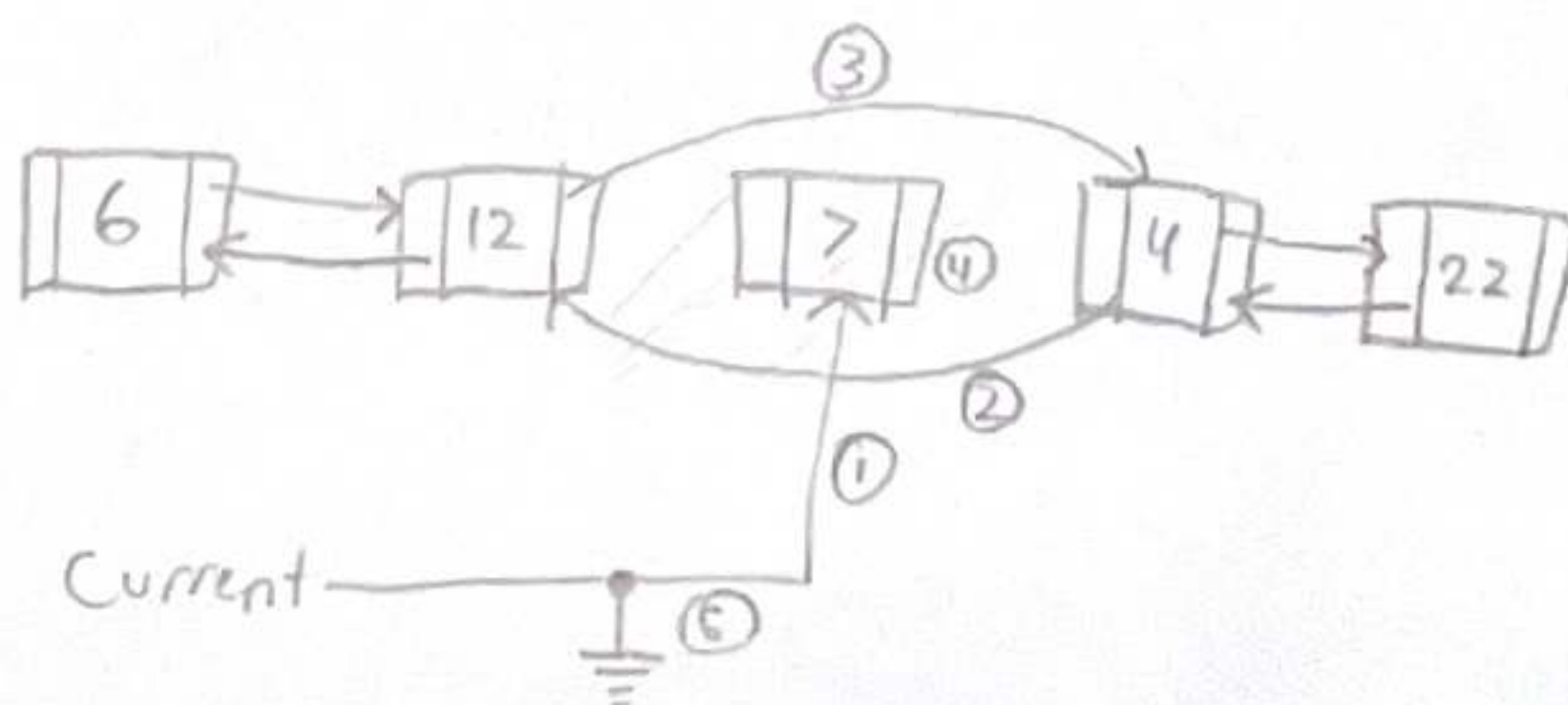
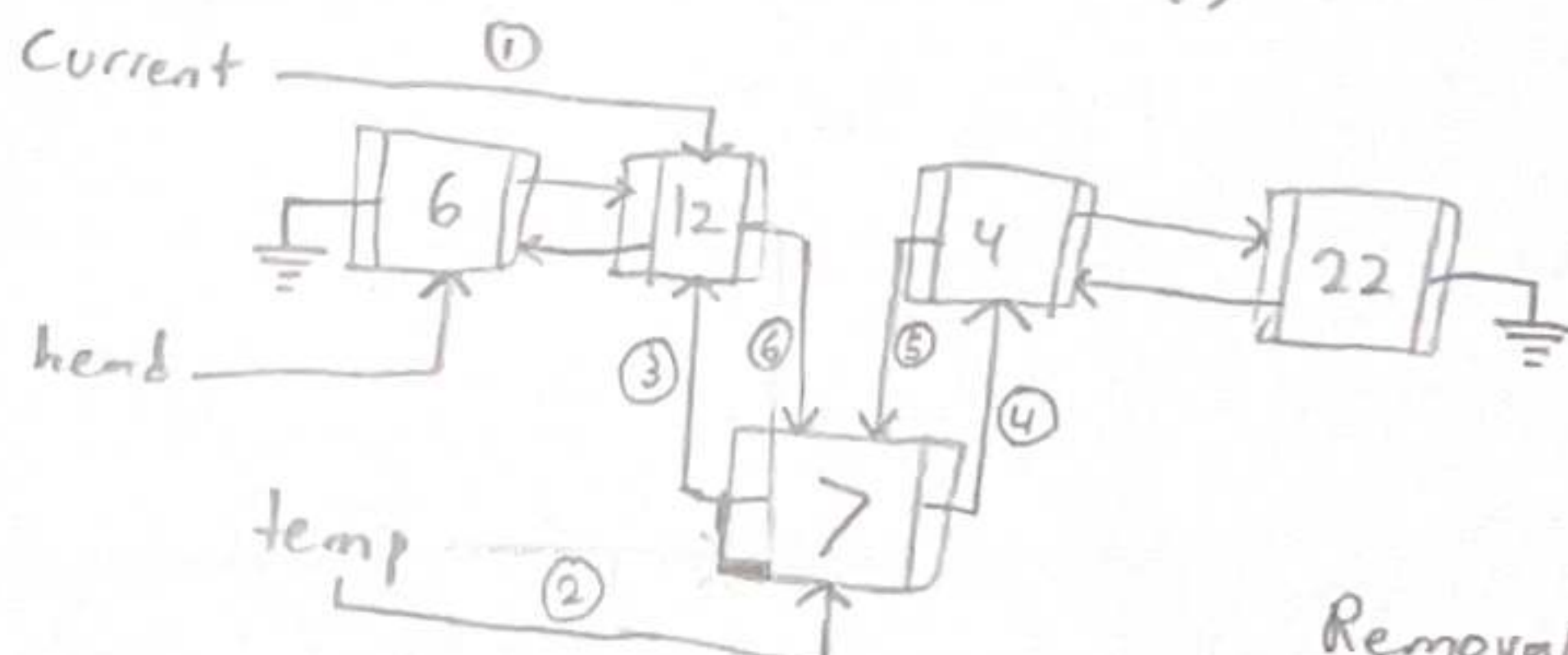
② temp = (Node*) malloc(sizeof(Node));
 temp->data = 7;

③ temp->prev = current;

④ temp->next = current->next

⑤ current->next->prev = temp

⑥ current->next = temp;



Removal Algorithm:

Node* current;

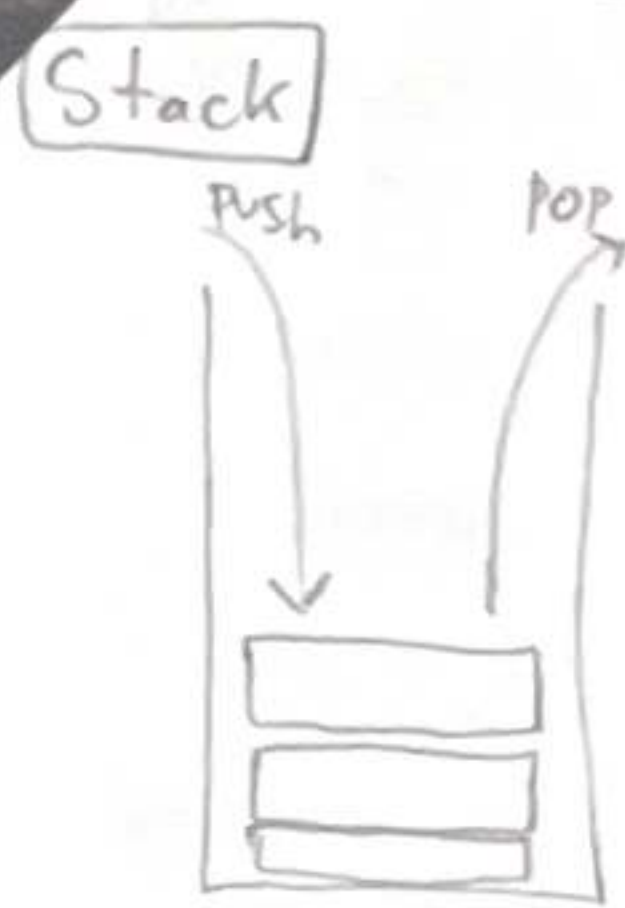
① // iterate current into place.....

② current->next->prev = current->prev;

③ current->prev->next = current->next;

④ free(current);

⑤ current = NULL;

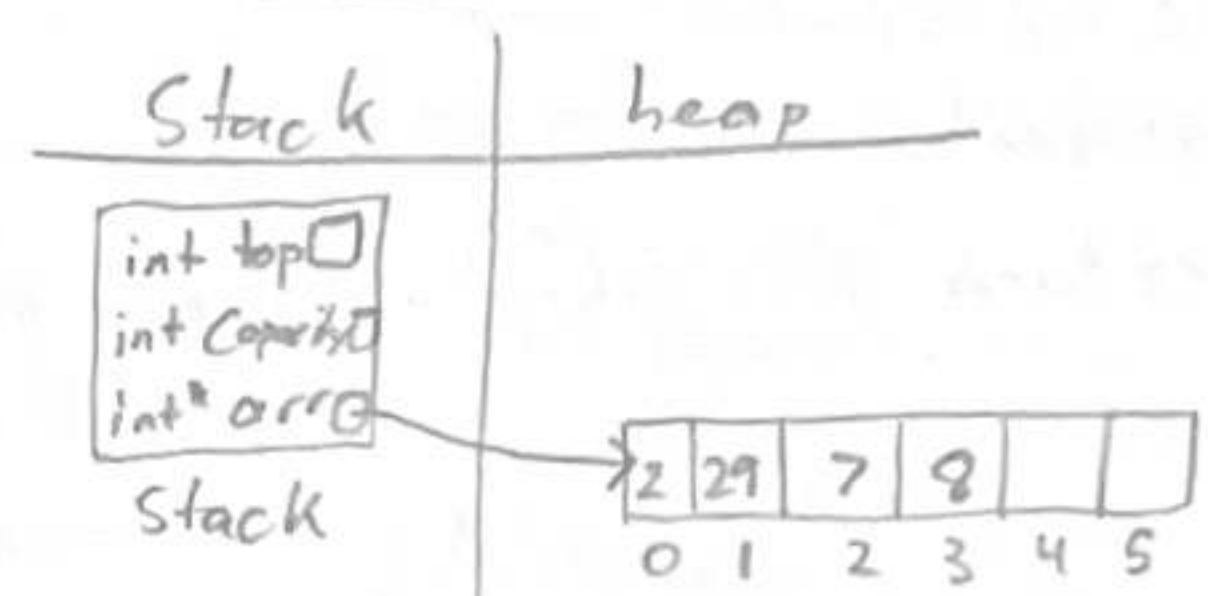


```
typedef struct stack {
    int top;
    int capacity;
    int* arr;
} Stack;
```

- Operations:**
- push** insert an item at top of the stack
 - Pop** remove and return top item
 - peek** return top item
 - isEmpty** does the stack contain any items

-Can use arrays or Linked Lists

Structure



top, index of first free space
capacity, array size

Queue

-First in First out

```
typedef struct {
    int front;
    int num;
    int capacity;
    int* arr;
} Queue;
```

Commands

- Enqueue** → inserts an item to the back of the queue
- Dequeue** → removes an item from the front of the queue

Circular array:

$$\text{Back} = (\text{front} + \text{num}) \% \text{Capacity}$$

$$\text{front} = (\text{front} + 1) \% \text{Capacity}$$

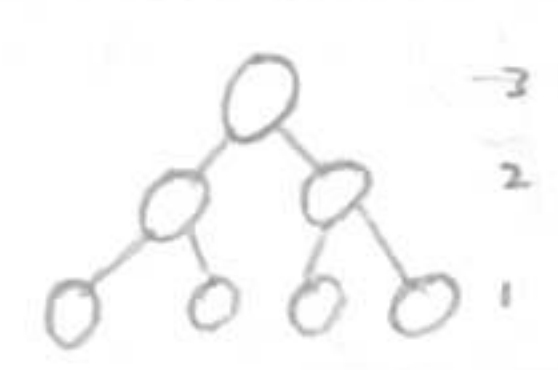
Terminology

- Leaf** → node with no children
- degree** → # of children a node has
- Branching Factor** → max degree any node can have

BST Efficiency

- Start at root
- if $\text{key} < \text{node}$ go left
- if $\text{key} > \text{node}$ go right

Perfect/Complete Binary Tree

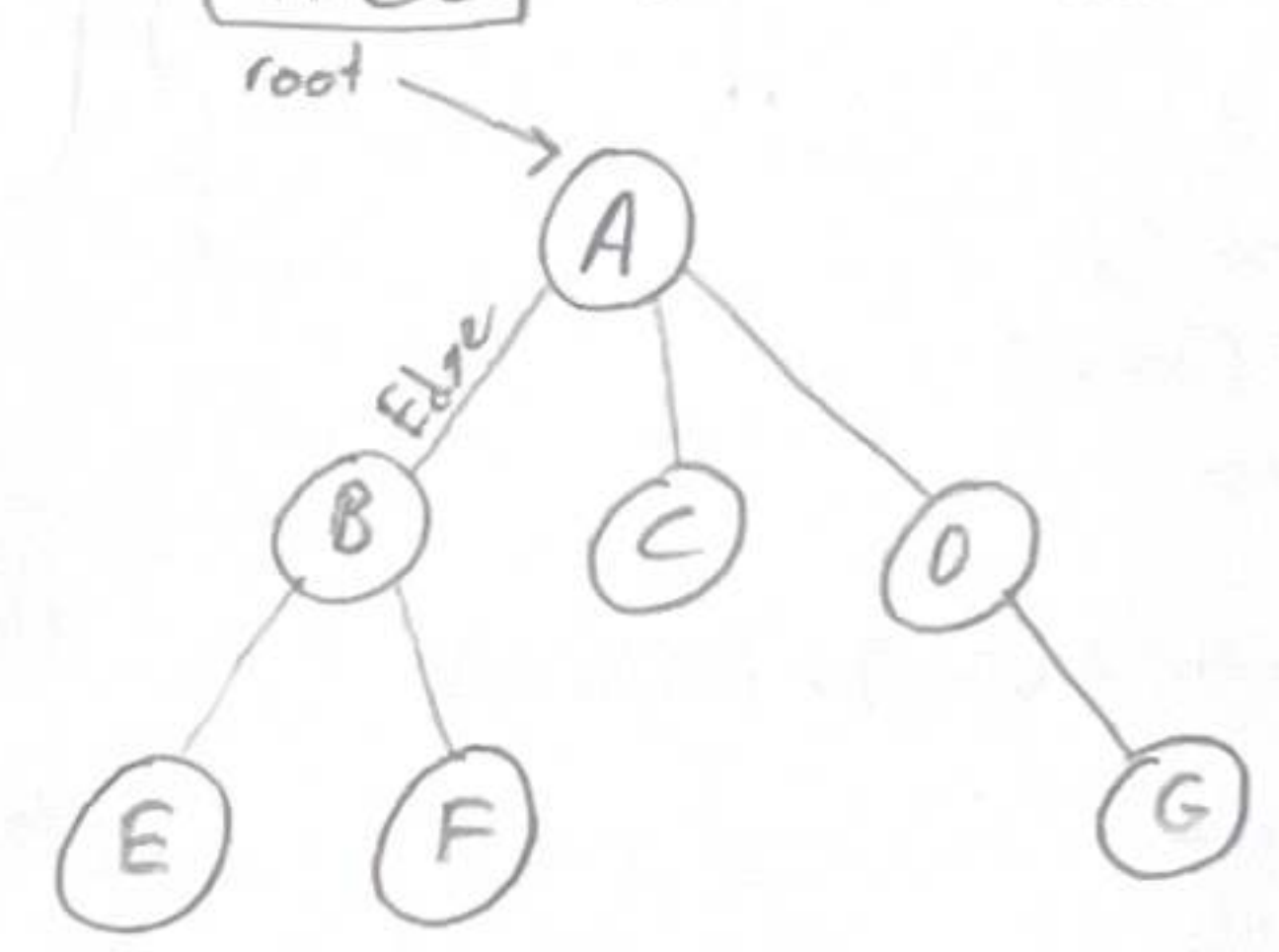


- a perfect tree with h levels has $2^h - 1$ nodes

Criteria (Complete):

- leaves are at most on two different heights
- second to bottom level is completely filled
- The leaves on the bottom are as far to the left as possible

Tree (Data Structure)



Traversals

- Pre-Order
- In-order
- Post-order

Recursion

minimum 2 cases:

- ① Base case, exit
- ② Recursive case, recursive call

Common uses

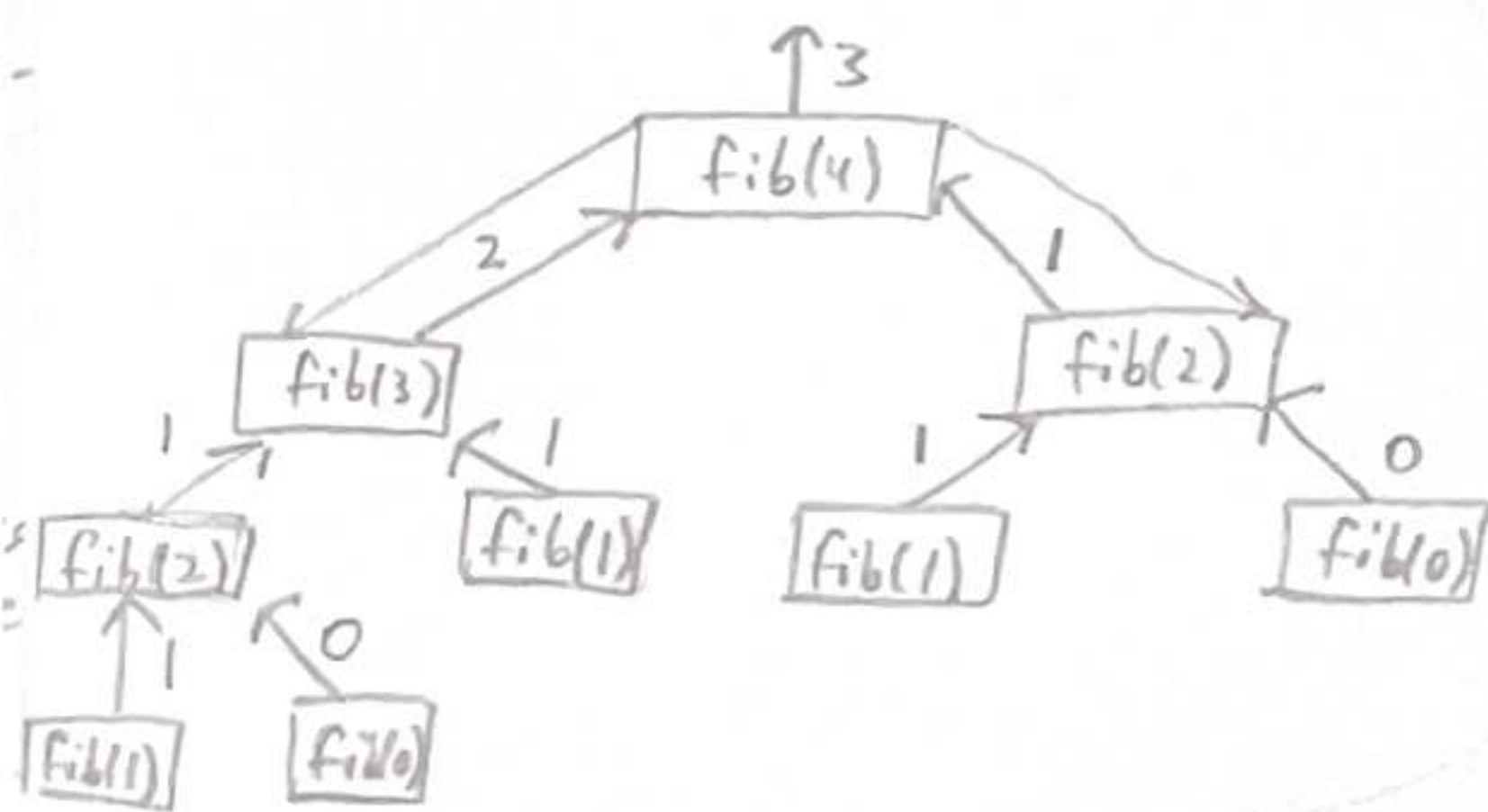
Fibonacci:

```
int fib(int n)
{
    if (n <= 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}
```

Factorial:

```
int factorial(int n)
{
    if (n <= 0)
        return 1;
    else
        return (n * factorial(n-1));
}
```

fib visualization:



BST Search

```
int Search(BNode* nd, int key)
{
    if (nd == null) return FALSE;
    else if (nd->data == key) return TRUE;
    else
    {
        if (key < nd->data)
            return Search(nd->left, key);
        else return Search(nd->right, key);
    }
}
```

BST Insertion

```
BNode* insert(BNode* nd, int key)
{
    if (nd == Null) {
        BNode* newnode = (BNode*) malloc(sizeof(BNode));
        newnode->data = key;
        newnode->left = Null;
        newnode->right = Null;
        return newnode;
    }
    else {
        if (key < nd->data)
            nd->left = insert(nd->left, key);
        else nd->right = insert(nd->right, key);
        return nd;
    }
}
```

Heaps

- Binary Tree with

2 properties:

- ① Complete, All levels except the bottom are completely filled in and the bottom level has leaves as far to the left as possible
- ② Partially ordered, its either
a max heap $\rightarrow$ every node has a larger value than its children
or a min heap $\rightarrow$ every node has a smaller value than its children

Implementation

Option 1: Array

• Index nodes from top to bottom left to right. Completeness ensures no gaps in the array



Accessing

• At node i

① parent node $\rightarrow \text{arr}[(i-1)/2]$

② Left child $\rightarrow \text{arr}[2*i+1]$

③ Right child $\rightarrow \text{arr}[2*i+2]$

Structure (MinHeap)

```
typedef struct MinHeap {
    int size;
    int capacity;
    int *arr;
} MinHeap;
```

Algorithms

Binary Search $\longrightarrow O(\log n)$

Sequential Search $\longrightarrow O(n)$

Tree Traversal $\longrightarrow O(n)$

Selection Sort $\longrightarrow O(n^2)$

Merge Sort $\longrightarrow O(n \log n)$

Insertion Sort $\longrightarrow O(n^2)$

Heap Sort $\longrightarrow O(n \log n)$

Hash Table

Handle Collisions

② Chaining.

Linked List or

① Open addressing, Tree Structure

look for a new array

position. How:

① linear probing

② Quadratic probing

③ double Hashing

Linear Probing

- Just proceed forward
to the next open space

Quadratic probing

- instead of proceeding one
spot at a time, search
spot, then $spot + 1$

$spot + 4$

$spot + 9$

$spot + 16$

Double Hashing

- Have a second hash
function to determine
the offset